

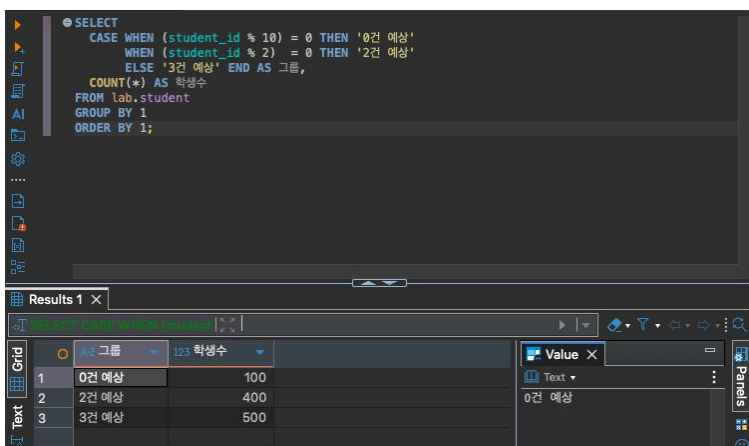
종합실습

CampusHub — JOIN + 서브쿼리 + 윈도우함수 복합 쿼리 실습 (skala_db / lab
스키마)

이름 : 허서연

1. 사전 검증

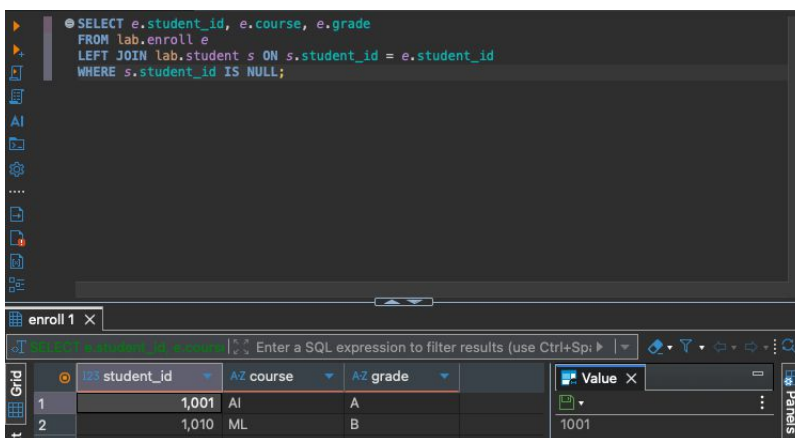
쿼리 실행 전 실습 데이터 확인



```
SELECT
  CASE WHEN (student_id % 10) = 0 THEN '0건 예상'
        WHEN (student_id % 2) = 0 THEN '2건 예상'
        ELSE '3건 예상' END AS 그룹,
  COUNT(*) AS 학생수
FROM lab.student
GROUP BY 1
ORDER BY 1;
```

그룹	학생수
0건 예상	100
2건 예상	400
3건 예상	500

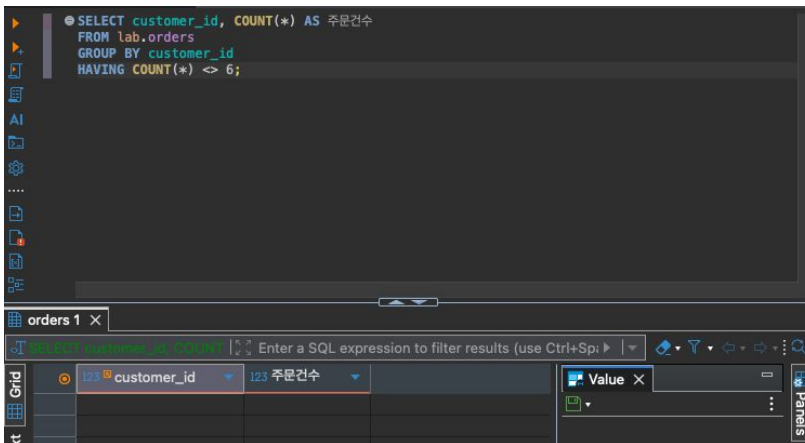
수강 건수 분포 — 0건 100명 / 2건 400명 / 3건 500명



```
SELECT e.student_id, e.course, e.grade
FROM lab.enroll e
LEFT JOIN lab.student s ON s.student_id = e.student_id
WHERE s.student_id IS NULL;
```

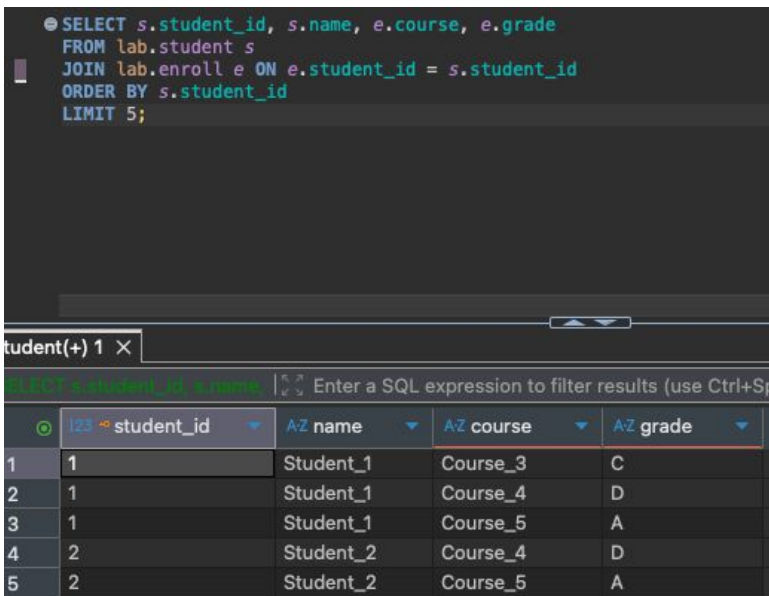
student_id	course	grade
1001	AI	A
1010	ML	B

고아 수강 확인 — student_id 1001(AI), 1010(ML) 2건



고객별 주문건수 이상치 확인

문항 1 — INNER JOIN



student_id, name, course, grade — 요구된 과목/성적 + 식별용 컬럼만포함

문항 2 — LEFT JOIN (모든 학생 기준)

```

SELECT s.student_id, s.name, e.course, e.grade
FROM lab.student s
LEFT JOIN lab.enroll e ON e.student_id = s.student_id
ORDER BY s.student_id
LIMIT 5;

```

Grid	123 student_id	AZ name	AZ course	AZ grade
1	1	Student_1	Course_3	C
2	1	Student_1	Course_4	D
3	1	Student_1	Course_5	A
4	2	Student_2	Course_4	D
5	2	Student_2	Course_5	A

LIMIT 없이 실행 시 2,400행 — LEFT JOIN 결과(매칭 2,300 + 비매칭 100)

```

SELECT s.student_id, s.name, e.course, e.grade
FROM lab.student s
LEFT JOIN lab.enroll e ON e.student_id = s.student_id
WHERE e.course IS NULL
ORDER BY s.student_id
LIMIT 5;

```

Grid	123 student_id	AZ name	AZ course	AZ grade
1	10	Student_10	[NULL]	[NULL]
2	20	Student_20	[NULL]	[NULL]
3	30	Student_30	[NULL]	[NULL]
4	40	Student_40	[NULL]	[NULL]
5	50	Student_50	[NULL]	[NULL]

[검증용 추가 쿼리] WHERE e.course IS NULL — 수강 0건 학생만 걸러서 NULL 처리 확인

문항 3 — RIGHT JOIN

```

SELECT e.student_id, e.course, e.grade, s.name
FROM lab.student s
RIGHT JOIN lab.enroll e ON e.student_id = s.student_id
ORDER BY e.course
LIMIT 5;

```

	student_id	course	grade	name
1	1001	AI	A	[NULL]
2	37	Course_1	A	Student_37
3	17	Course_1	A	Student_17
4	18	Course_1	A	Student_18
5	19	Course_1	A	Student_19

수강을 기준으로 right join

```

SELECT e.course, e.grade, s.name
FROM lab.student s
RIGHT JOIN lab.enroll e ON e.student_id = s.student_id
WHERE s.student_id IS NULL;

```

	course	grade	name
1	AI	A	[NULL]
2	ML	B	[NULL]

[검증용 추가 쿼리] WHERE s.student_id IS NULL — 고아 수강(AI, ML) 2건이 정확히 나옴, name도 NULL 확인

student_id=1001(AI, name NULL) 행이 맨 위에 정렬되어 나온 것도 확인했습니다. enroll 테이블은 자체 PK가 없어서 (student_id, course) 조합이 사실상의 식별자인데, 이제 student_id가 포함되어 어떤 학생의 어떤 수강 기록인지 완전히 특정할 수 있게 되었습니다.

문항 4 — FULL JOIN

```

SELECT COALESCE(s.student_id, e.student_id) AS student_id, s.name, e.course, e.grade
FROM lab.student s
FULL JOIN lab.enroll e ON e.student_id = s.student_id
ORDER BY 1
LIMIT 5;

```

	student_id	name	course	grade
1	1	Student_1	Course_3	C
2	1	Student_1	Course_5	A
3	1	Student_1	Course_4	D
4	2	Student_2	Course_5	A
5	2	Student_2	Course_4	D

COALESCE(s.student_id, e.student_id)로 양쪽 키를 병합, 학생/수강 모두 포함

문항 5 — 한 번도 수강하지 않은 학생 NOT EXISTS

```

SELECT s.student_id, s.name
FROM lab.student s
WHERE NOT EXISTS (
  SELECT 1 FROM lab.enroll e WHERE e.student_id = s.student_id
)
ORDER BY s.student_id
LIMIT 5;

```

student_id	name
10	Student_10
20	Student_20
30	Student_30
40	Student_40
50	Student_50

문항 6 — 한 과목 이상 수강 (중복 제거) DISTINCT

```

SELECT DISTINCT s.student_id, s.name
FROM lab.student s
JOIN lab.enroll e ON e.student_id = s.student_id
ORDER BY s.student_id
LIMIT 5;

```

student_id	name
1	Student_1
2	Student_2
3	Student_3
4	Student_4
5	Student_5

DISTINCT s.student_id, s.name — 중복 제거 정상 적용

문항 7 — 고객별 주문건수/총액 SUM

```
SELECT o.customer_id, COUNT(*) AS 주문건수, SUM(o.amount) AS 총액
FROM lab.orders o
GROUP BY o.customer_id
ORDER BY o.customer_id
LIMIT 5;
```

customer_id	주문건수	총액
1	6	16,530
2	6	12,608.06
3	6	12,686.12
4	6	12,764.18
5	6	12,842.24

customer_id, 주문건수, 총액 — 전 고객 6건씩 정상 확인

문항 8 — 총액 상위 10 명 LIMIT 10

```
SELECT o.customer_id, SUM(o.amount) AS 총액
FROM lab.orders o
GROUP BY o.customer_id
ORDER BY 총액 DESC
LIMIT 10;
```

customer_id	총액
154	19,467.18
153	19,389.12
152	19,311.06
151	19,233
150	19,154.94
149	19,076.88
148	18,998.82
147	18,920.76
146	18,842.7
145	18,764.64

ORDER BY 총액 DESC LIMIT 10 — customer_id 154부터 145까지 내림차순 10건

문항 9 — 모든 직원과 매니저 이름 (Self Join)

```

SELECT e.emp_id, e.name AS 직원, m.name AS 매니저
FROM lab.emp e
LEFT JOIN lab.emp m ON m.emp_id = e.manager_id
ORDER BY e.emp_id
LIMIT 5;

```

	emp_id	직원	매니저
1	1	CEO	[NULL]
2	2	Mgr_2	CEO
3	3	Mgr_3	CEO
4	4	Mgr_4	CEO
5	5	Mgr_5	CEO

emp를 자기 자신과 LEFT JOIN — CEO 행의 매니저가 NULL로 정상 표시

문항 10 — 모든 학생 기준 과목 분포 (LEFT JOIN + 집계)

```

SELECT s.student_id, s.name, COUNT(e.course) AS 수강과목수
FROM lab.student s
LEFT JOIN lab.enroll e ON e.student_id = s.student_id
GROUP BY s.student_id, s.name
ORDER BY s.student_id
LIMIT 5;

```

	student_id	name	수강과목수
1	1	Student_1	3
2	2	Student_2	2
3	3	Student_3	3
4	4	Student_4	2
5	5	Student_5	3

COUNT(e.course)로 학생별 수강 과목 수 집계 — COUNT() 아닌 COUNT(course)라 0건 학생도 정확히 0으로 집계*

문항 11 — DB 과목을 듣지 않은 학생 NOT EXIST

```

SELECT s.student_id, s.name
FROM lab.student s
WHERE NOT EXISTS (
  SELECT 1 FROM lab.enroll e
  WHERE e.student_id = s.student_id AND e.course = 'DB'
)
ORDER BY s.student_id
LIMIT 5;

```

student_id	AZ name
1	Student_1
2	Student_2
3	Student_3
4	Student_4
5	Student_5

NOT EXISTS + course='DB' 조건 — Anti-Join 패턴 정확히 적용

문항 12 — course_owner 매핑 테이블 + 리포트

```

CREATE TABLE lab.course_owner (
  course VARCHAR(50) PRIMARY KEY,
  manager_id INT REFERENCES lab.emp(emp_id)
);

INSERT INTO lab.course_owner (course, manager_id)
SELECT c.course, m.emp_id
FROM (
  SELECT DISTINCT course, ROW_NUMBER() OVER (ORDER BY course) AS rn
  FROM lab.enroll
) c
JOIN (
  SELECT emp_id, ROW_NUMBER() OVER (ORDER BY emp_id) AS rn
  FROM lab.emp WHERE name LIKE 'Mgr_%'
) m ON m.rn = ((c.rn - 1) % 10) + 1
ON CONFLICT (course) DO NOTHING;

SELECT co.course,
  COUNT(DISTINCT e.student_id) AS 수강인원,
  mgr.name AS 책임매니저
FROM lab.course_owner co
JOIN lab.emp mgr ON mgr.emp_id = co.manager_id
LEFT JOIN lab.enroll e ON e.course = co.course
GROUP BY co.course, mgr.name
ORDER BY co.course
LIMIT 5;

```

course	수강인원	AZ 책임매니저
AI	1	Mgr_2
Course_1	144	Mgr_5
Course_10	96	Mgr_3
Course_11	144	Mgr_9
Course_12	48	Mgr_7

CREATE TABLE → 라운드로빈 매핑 INSERT(ON CONFLICT 방어 포함)

문항 13 — 학생×과목 전체 조합 (CROSS JOIN), 샘플 100건


```

SELECT s.student_id, s.name, s.major, s.gpa
FROM lab.student s
WHERE s.gpa > (SELECT AVG(gpa) FROM lab.student)
ORDER BY s.student_id
LIMIT 5;

```

	student_id	name	major	gpa
1	15	Student_15	CS	3.5
2	16	Student_16	EE	3.6
3	17	Student_17	ME	3.7
4	18	Student_18	CE	3.8
5	19	Student_19	BIO	3.9

student_id부터 gpa까지 4개 컬럼, WHERE 서브쿼리로 전체 평균과 비교

문항 16 — 자기 학과 평균 GPA 보다 높은 학생 (상관 서브쿼리)

```

SELECT s.student_id, s.name, s.major, s.gpa
FROM lab.student s
WHERE s.gpa > (
  SELECT AVG(s2.gpa) FROM lab.student s2 WHERE s2.major = s.major
)
ORDER BY s.student_id
LIMIT 5;

```

	student_id	name	major	gpa
1	15	Student_15	CS	3.5
2	16	Student_16	EE	3.6
3	17	Student_17	ME	3.7
4	18	Student_18	CE	3.8
5	19	Student_19	BIO	3.9

문항 17 — 수강 기록이 있는 학생만 (EXISTS)

```

SELECT s.student_id, s.name
FROM lab.student s
WHERE EXISTS (
  SELECT 1 FROM lab.enroll e WHERE e.student_id = s.student_id
)
ORDER BY s.student_id
LIMIT 5;

```

	student_id	name
1	1	Student_1
2	2	Student_2
3	3	Student_3
4	4	Student_4
5	5	Student_5

문항 18 — 한 번도 수강하지 않은 학생

```
SELECT s.student_id, s.name
FROM lab.student s
WHERE NOT EXISTS (
  SELECT 1 FROM lab.enroll e WHERE e.student_id = s.student_id
)
ORDER BY s.student_id
LIMIT 5;
```

student_id	name
10	Student_10
20	Student_20
30	Student_30
40	Student_40
50	Student_50

문항 5(Anti-Join)와 동일 로직을 NOT EXISTS로 재작성

문항 19 — HR 학과 학생 비교 데모

```
SELECT s.student_id, s.name, s.major, s.gpa,
       (SELECT ROUND(AVG(gpa), 2) FROM lab.student WHERE major = 'HR') AS hr_avg_gpa
FROM lab.student s
WHERE s.major = 'HR'
ORDER BY s.student_id
LIMIT 5;
```

student_id	name	major	gpa	hr_avg_gpa
981	Student_981	HR	4.1	3.4
982	Student_982	HR	4.2	3.4
983	Student_983	HR	4.3	3.4
984	Student_984	HR	4.4	3.4
985	Student_985	HR	4.5	3.4

HR 학과 개별 GPA + HR 학과 평균(hr_avg_gpa)을 나란히 표시

문항 20 — CS 학과 또는 DB 과목 수강 학생

```
SELECT DISTINCT s.student_id, s.name, s.major
FROM lab.student s
WHERE s.major = 'CS'
   OR EXISTS (SELECT 1 FROM lab.enroll e WHERE e.student_id = s.student_id AND e.course = 'DB')
ORDER BY s.student_id
LIMIT 5;
```

student_id	name	major
5	Student_5	CS
10	Student_10	CS
15	Student_15	CS
19	Student_19	BIO
20	Student_20	CS

문항 21 — GROUP BY ROLLUP

```
SELECT
CASE WHEN GROUPING(t.major) = 1 THEN '전체' ELSE t.major END AS major,
CASE WHEN GROUPING(t.gpa_tier) = 1 THEN '소계' ELSE t.gpa_tier END AS gpa_tier,
COUNT(*) AS 인원
FROM (
SELECT
student_id,
major,
CASE
WHEN gpa < 3.0 THEN '3.0 미만'
WHEN gpa <= 3.5 THEN '3.0~3.5'
ELSE '3.5 초과'
END AS gpa_tier
FROM lab.student
) t
GROUP BY ROLLUP(t.major, t.gpa_tier)
ORDER BY GROUPING(t.major), t.major, GROUPING(t.gpa_tier), t.gpa_tier;
```

Results 1 X

Enter a SQL expression to filter results (use Ctrl+Space)

	AZ major	AZ gpa_tier	123 인원	
1	BIO	3.0 미만	66	
2	BIO	3.0~3.5	33	
3	BIO	3.5 초과	97	
4	BIO	소계	196	
5	CE	3.0 미만	66	
6	CE	3.0~3.5	33	
7	CE	3.5 초과	97	
8	CE	소계	196	

Value X

Text

BIO

GPA 구간 파생 컬럼 + ROLLUP(major, gpa_tier) + GROUPING()으로 '전체'/'소계' 라벨 처리

문항 22 — 데이터 이상 발견

```
WITH RECURSIVE org_tree AS (
SELECT emp_id, name, manager_id, 0 AS depth, name::TEXT AS path
FROM lab.emp
WHERE manager_id IS NULL
UNION ALL
SELECT e.emp_id, e.name, e.manager_id, t.depth + 1, t.path || ' > ' || e.name
FROM lab.emp e
JOIN org_tree t ON t.emp_id = e.manager_id
)
SELECT emp_id, name, depth, path
FROM org_tree
ORDER BY path
LIMIT 5;
```

org_tree 1 X

Enter a SQL expression to filter results (use Ctrl+Space)

	123 emp_id	AZ name	123 depth	AZ path	
1	1	CEO	0	CEO	
2	102	Dev_102	1	CEO > Dev_102	1
3	112	Dev_112	1	CEO > Dev_112	
4	12	Dev_12	1	CEO > Dev_12	
5	122	Dev_122	1	CEO > Dev_122	

Value X

WITH RECURSIVE org_tree — emp_id, name, depth, path

문항 23 — 학과별 GPA 순위 Top-3 (서브쿼리 + CTE)

```

SELECT student_id, name, major, gpa, rn, rnk, dense_rnk, total_in_major
FROM (
    SELECT student_id, name, major, gpa,
        ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id ASC) AS rn,
        RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
        DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS dense_rnk,
        COUNT(*) OVER (PARTITION BY major) AS total_in_major
    FROM lab.student
) t
WHERE rn <= 3
ORDER BY major, rn;

```

student_id	name	major	gpa	rn	rnk	total_in_major
29	Student_29	BIO	4.9	1	1	1
59	Student_59	BIO	4.9	2	1	1
89	Student_89	BIO	4.9	3	1	1
28	Student_28	CE	4.8	1	1	1
58	Student_58	CE	4.8	2	1	1
88	Student_88	CE	4.8	3	1	1
25	Student_25	CS	4.5	1	1	1
55	Student_55	CS	4.5	2	1	1
85	Student_85	CS	4.5	3	1	1
26	Student_26	EE	4.6	1	1	1
56	Student_56	EE	4.6	2	1	1
86	Student_86	EE	4.6	3	1	1
989	Student_989	HR	4.9	1	1	1
988	Student_988	HR	4.8	2	2	2
987	Student_987	HR	4.7	3	3	3
27	Student_27	ME	4.7	1	1	1
57	Student_57	ME	4.7	2	1	1
87	Student_87	ME	4.7	3	1	1

[재실행] 방식 1: 서브쿼리 — student_id 추가 반영, 18행(6개 학과 × 3명)

```

WITH ranked AS (
    SELECT student_id, name, major, gpa,
        ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id ASC) AS rn,
        RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
        DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS dense_rnk,
        COUNT(*) OVER (PARTITION BY major) AS total_in_major
    FROM lab.student
)
SELECT student_id, name, major, gpa, rn, rnk, dense_rnk, total_in_major
FROM ranked
WHERE rn <= 3
ORDER BY major, rn;

```

student_id	name	major	gpa	rn	rnk	total_in_major
29	Student_29	BIO	4.9	1	1	1
59	Student_59	BIO	4.9	2	1	1
89	Student_89	BIO	4.9	3	1	1
28	Student_28	CE	4.8	1	1	1
58	Student_58	CE	4.8	2	1	1
88	Student_88	CE	4.8	3	1	1
25	Student_25	CS	4.5	1	1	1
55	Student_55	CS	4.5	2	1	1
85	Student_85	CS	4.5	3	1	1
26	Student_26	EE	4.6	1	1	1
56	Student_56	EE	4.6	2	1	1
86	Student_86	EE	4.6	3	1	1
989	Student_989	HR	4.9	1	1	1
988	Student_988	HR	4.8	2	2	2
987	Student_987	HR	4.7	3	3	3
27	Student_27	ME	4.7	1	1	1
57	Student_57	ME	4.7	2	1	1

[재실행] 방식 2: CTE — 서브쿼리 방식과 완전히 동일한 결과, student_id 포함

문항 24 — LAG 성적 변화 추이

```

WITH scored AS (
  SELECT
    student_id,
    course,
    grade,
    CASE grade
      WHEN 'A' THEN 4
      WHEN 'B' THEN 3
      WHEN 'C' THEN 2
      WHEN 'D' THEN 1
    END AS score
  FROM lab.enroll
)
SELECT
  student_id,
  course,
  grade,
  score,
  LAG(score) OVER (PARTITION BY student_id ORDER BY course) AS prev_score,
  score - LAG(score) OVER (PARTITION BY student_id ORDER BY course) AS diff,
  CASE
    WHEN LAG(score) OVER (PARTITION BY student_id ORDER BY course) IS NULL THEN '이전 기록 없음'
    WHEN score - LAG(score) OVER (PARTITION BY student_id ORDER BY course) > 0 THEN '상승'
    WHEN score - LAG(score) OVER (PARTITION BY student_id ORDER BY course) < 0 THEN '하락'
    ELSE '유지'
  END AS trend,
  MAX(score) OVER (PARTITION BY student_id) - MIN(score) OVER (PARTITION BY student_id) AS score_range
FROM scored
ORDER BY student_id, course
LIMIT 5;

```

123 student_id	AZ course	AZ grade	123 score	123 prev_score	123 diff	AZ trend	123 score_range
1	Course_3	C	2	[NULL]	[NULL]	이전 기록 없음	3
1	Course_4	D	1	2	-1	하락	3
1	Course_5	A	4	1	3	상승	3
2	Course_4	D	1	[NULL]	[NULL]	이전 기록 없음	3
2	Course_5	A	4	1	3	상승	3

등급→점수 CASE 변환, LAG로 이전 과목 점수, diff/trend, score_range까지 전부 반영

문항 25 — 누적 주문금액 & 이동평균

```

SELECT
  order_id,
  customer_id,
  amount,
  SUM(amount) OVER (
    ORDER BY order_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total,
  AVG(amount) OVER (
    ORDER BY order_id
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_avg_3,
  SUM(amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS customer_running_total
FROM lab.orders
ORDER BY order_id
LIMIT 5;

```

123 order_id	123 customer_id	123 amount	123 running_total	123 moving_avg_3	123 customer_running_total
1	2	18.01	18.01	18.01	18.01
2	3	31.02	49.03	24.515	31.02
3	4	44.03	93.06	31.02	44.03
4	5	57.04	150.1	44.03	57.04
5	6	70.05	220.15	57.04	70.05

running_total(전체 누적), moving_avg_3(3건 이동평균), customer_running_total(고객별 누적) 세 지표 모두 반영

```

WITH running AS (
  SELECT
    order_id,
    SUM(amount) OVER (
      ORDER BY order_id
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total,
    SUM(amount) OVER () AS grand_total
  FROM lab.orders
)
SELECT order_id, running_total, grand_total,
  ROUND(running_total / grand_total * 100, 2) AS pct
FROM running
WHERE running_total > grand_total * 0.5
ORDER BY order_id
LIMIT 1;

```

ers 1 X

running AS (SELECT order_id, SUM(amount) OVER (ORDER BY order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total, SUM(amount) OVER () AS grand_total FROM lab.orders)

123 order_id	123 running_total	123 grand_total	123 pct
1,503	3,686,585.56	7,365,985	50.05

누적합이 전체의 50%를 초과하는 첫 order_id = 1,503 (pct 50.05%)